

Diseño de experimentos para clasificación de textos

Alejandra Paola Castillo Gallegos

Matrícula: 1801137

Materia: Procesamiento y Clasificación de Datos

Maestría en Ciencia de Datos

Facultad de Ciencias Físico Matemáticas, Universidad Autónoma de Nuevo León

I. INTRODUCCIÓN

La clasificación automática de textos constituye una de las principales tareas dentro del Procesamiento de Lenguaje Natural (NLP), ya que permite asignar documentos o mensajes a categorías previamente definidas a partir de su contenido. Entre sus aplicaciones se encuentran la detección de spam, el análisis de sentimiento, la clasificación temática de documentos y la identificación automática de contenido no deseado.

El desempeño de un sistema de clasificación textual no depende únicamente del algoritmo de aprendizaje utilizado. La forma en que el texto se transforma en una representación numérica, así como los hiperparámetros utilizados durante el entrenamiento, pueden producir diferencias importantes en la capacidad predictiva del modelo.

En este trabajo se desarrolla un diseño experimental para comparar distintas combinaciones de técnicas de vectorización, algoritmos de clasificación e hiperparámetros. El problema seleccionado corresponde a la clasificación de mensajes SMS en dos categorías: mensajes legítimos, denominados ham, y mensajes no deseados o spam.

Se comparan representaciones basadas en conteos de términos y TF-IDF, junto con tres algoritmos ampliamente utilizados para clasificación de textos: Regresión Logística, Multinomial Naive Bayes y Support Vector Machine lineal. Para seleccionar las mejores configuraciones se utiliza búsqueda sistemática de hiperparámetros y validación cruzada estratificada.

El objetivo final es identificar la configuración que proporcione el mejor equilibrio entre precisión, sensibilidad y capacidad general de clasificación, prestando especial atención al reconocimiento de mensajes spam.

II. DESCRIPCIÓN DE DATOS

El conjunto de datos utilizado corresponde a una colección de mensajes SMS previamente etiquetados para un problema de clasificación binaria. Cada observación contiene el texto de un mensaje y su respectiva categoría.

La variable objetivo presenta dos posibles valores:

- **Ham:** mensaje legítimo o no deseado.
- **Spam:** mensaje identificado como publicidad, fraude, promoción o comunicación no solicitada.

Antes del modelado se realizó un análisis exploratorio para determinar el número de observaciones, presencia de valores faltantes, registros duplicados y distribución de las clases.

El análisis de frecuencias permitió identificar un desbalance entre las categorías, debido a que la cantidad de mensajes legítimos es considerablemente mayor que la cantidad de mensajes spam. Esta característica se tomó en consideración durante el diseño experimental, por lo que además de la exactitud global (accuracy) se utilizaron métricas como precisión, recall y F1-score.

El texto fue sometido a un proceso de normalización que incluyó la conversión a minúsculas, eliminación de URLs, correos electrónicos, símbolos especiales y espacios repetidos. Posteriormente se eliminaron registros duplicados antes de realizar la separación entre los conjuntos de entrenamiento y prueba, evitando así posibles problemas de fuga de información.

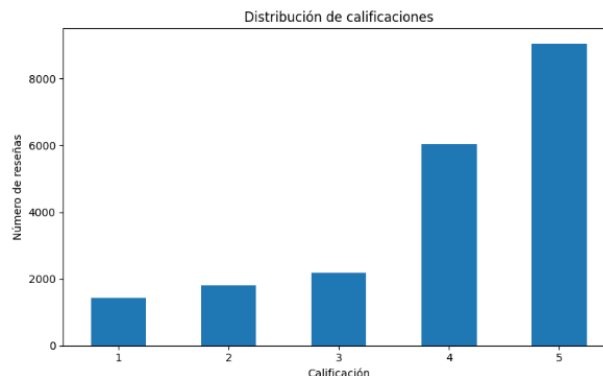


Fig. 1: Distribución de clases

III. METODOLOGÍA

El procedimiento experimental se estructuró como un problema supervisado de clasificación binaria. El conjunto de datos fue dividido en un 80% para entrenamiento y un 20% para prueba. La división se realizó mediante muestreo estratificado para conservar aproximadamente la misma proporción de mensajes ham y spam en ambos subconjuntos.[learn Developers, 2026]

A. Representación del texto

Se evaluaron dos enfoques principales de representación numérica.

El primero fue **CountVectorizer**, que representa cada documento mediante la frecuencia de aparición de los términos. Este método genera una matriz documento-término basada directamente en conteos.

El segundo fue **TF-IDF (Term Frequency–Inverse Document Frequency)**, que asigna un peso a cada término considerando simultáneamente su frecuencia dentro de un documento y su rareza dentro del corpus completo.

El objetivo es disminuir la importancia de palabras muy frecuentes y aumentar el peso relativo de términos más informativos.

Además, se evaluaron diferentes rangos de n-gramas, incluyendo unigramas y combinaciones de unigramas y bigramas.

B. Modelos de clasificación

Se compararon tres algoritmos.

- **Regresión Logística.** Modelo lineal probabilístico utilizado ampliamente en clasificación binaria. Se evaluaron diferentes valores del hiperparámetro de regularización C .
- **Multinomial Naive Bayes.** Clasificador probabilístico especialmente adecuado para datos basados en frecuencias de palabras. Se modificó el parámetro de suavizado α .
- **Support Vector Machine lineal.** Modelo que busca construir un hiperplano capaz de separar las clases maximizando el margen entre ellas. Se evaluaron distintos valores del parámetro de regularización C .

C. Diseño experimental

El experimento consideró tres factores principales: método de vectorización, algoritmo de clasificación e hiperparámetros.

Las combinaciones fueron evaluadas mediante Grid-SearchCV, permitiendo explorar sistemáticamente distintas configuraciones. Para estimar el desempeño de forma más robusta se utilizó validación cruzada estratificada de cinco particiones.

La selección de modelos se realizó utilizando F1 macro como función de evaluación durante la validación cruzada. Posteriormente, cada mejor configuración fue evaluada sobre el conjunto de prueba independiente.

D. Métricas

Se utilizaron las siguientes métricas:

- *Accuracy.*
- *Precision para la clase spam.*
- *Recall para la clase spam.*
- *F1-score para la clase spam.*
- *F1 macro durante validación cruzada.*
- *Matriz de confusión.*

Debido al desbalance existente entre las clases, el F1-score y el recall de la clase spam se consideraron especialmente importantes para determinar la calidad del modelo.

IV. DISEÑO DE EXPERIMENTOS

Con el propósito de comparar de manera sistemática diferentes estrategias para la clasificación automática de mensajes de texto, se planteó un diseño experimental basado en tres componentes principales: el método de representación del texto, el algoritmo de clasificación y los hiperparámetros asociados a cada configuración.

Se evaluaron dos métodos de vectorización. El primero fue **CountVectorizer**, basado en la frecuencia de aparición de los términos dentro de cada documento. El segundo fue **TF-IDF (Term Frequency–Inverse Document Frequency)**, que pondera los términos considerando tanto su frecuencia dentro de un documento como su frecuencia relativa dentro del corpus completo.

Para la etapa de clasificación se utilizaron tres algoritmos supervisados: **Regresión Logística, Multinomial Naive Bayes y Support Vector Machine lineal (LinearSVC)**. La combinación de los dos esquemas de vectorización con los tres clasificadores dio lugar a seis familias principales de experimentos.

TABLE I: Configuraciones generales del diseño experimental

Experimento	Vectorización	Modelo	Hiperparámetros evaluados
E1	CountVectorizer	Logistic Regression	C , min_df, ngram_range
E2	TF-IDF	Logistic Regression	C , min_df, ngram_range, max_df
E3	CountVectorizer	MultinomialNB	α , min_df, ngram_range
E4	TF-IDF	MultinomialNB	α , min_df, ngram_range, max_df
E5	CountVectorizer	LinearSVC	C , min_df, ngram_range
E6	TF-IDF	LinearSVC	C , min_df, ngram_range, max_df

La búsqueda de hiperparámetros se realizó mediante **Grid-SearchCV**, utilizando validación cruzada estratificada de cinco particiones. Este procedimiento permitió evaluar cada configuración sobre diferentes subconjuntos del conjunto de entrenamiento y reducir la dependencia de una única partición aleatoria de los datos.

En total se evaluaron **96 combinaciones distintas de modelos e hiperparámetros**. Debido al uso de validación cruzada de cinco particiones, esto representó un total de **480 ajustes o entrenamientos internos de modelos** durante el proceso de optimización.

La distribución de configuraciones fue la siguiente: 12 combinaciones para CountVectorizer con Regresión Logística, 24 para TF-IDF con Regresión Logística, 12 para CountVectorizer con Multinomial Naive Bayes, 12 para TF-IDF con Multinomial Naive Bayes, 12 para CountVectorizer con LinearSVC y 24 para TF-IDF con LinearSVC.

Como métrica de optimización durante la validación cruzada se utilizó el **F1 macro**, debido a que esta métrica calcula el desempeño considerando ambas clases con la misma importancia, independientemente de su frecuencia. Esta decisión resulta especialmente relevante debido al desbalance observado en el conjunto de datos, donde aproximadamente el 86.6% de los mensajes corresponde a la clase ham y el 13.4% a la clase spam.

Para la evaluación final se mantuvo separado un conjunto de prueba equivalente al 20% de los datos, compuesto por **1,115 mensajes**, mientras que **4,457 observaciones** fueron utilizadas para entrenamiento y optimización. La partición se realizó mediante estratificación para conservar aproximadamente la misma proporción de clases en ambos subconjuntos.

Los hiperparámetros analizados incluyeron distintos valores de regularización C para Regresión Logística y LinearSVC;

valores de suavizado α para Multinomial Naive Bayes; frecuencias mínimas de aparición mediante $\min_df$; representaciones mediante unigramas y combinaciones de unigramas con bigramas; y, para algunas configuraciones de TF-IDF, restricciones sobre el número máximo de características mediante $\max_features$.

Este diseño permitió analizar no solamente qué algoritmo presenta mejor desempeño, sino también determinar cómo la representación de los textos y la configuración de hiperparámetros influyen sobre la capacidad predictiva del sistema.

V. RESULTADOS

A. Comparación general de los modelos

Después de completar el proceso de búsqueda de hiperparámetros mediante validación cruzada, se seleccionó la mejor configuración de cada una de las seis familias experimentales. Posteriormente, estos modelos fueron evaluados sobre el conjunto de prueba independiente.

Los resultados obtenidos se presentan en la Tabla 2.

TABLE II: Comparación de los mejores modelos sobre el conjunto de prueba

Modelo	CV F1 macro	Accuracy	Precision
TF-IDF + LinearSVC	0.9674	0.9865	0.98
TF-IDF + Logistic Regression	0.9639	0.9848	0.98
CountVectorizer + MultinomialNB	0.9690	0.9830	0.95
TF-IDF + MultinomialNB	0.9699	0.9830	0.9924
CountVectorizer + Logistic Regression	0.9590	0.9812	0.963
CountVectorizer + LinearSVC	0.9642	0.9803	0.9635

Los seis modelos presentaron niveles elevados de exactitud, con valores de accuracy superiores al 98%. Sin embargo, debido al desbalance existente entre las categorías, el análisis no debe limitarse únicamente a esta métrica. Las diferencias resultan más evidentes al observar el recall y el F1-score específicamente para la clase spam.

La configuración **TF-IDF + LinearSVC** obtuvo el mejor desempeño global sobre el conjunto de prueba, alcanzando un **accuracy de 0.9865**, equivalente aproximadamente al 98.65% de clasificación correcta. Para la identificación específica de spam obtuvo una **precision de 0.9855**, un **recall de 0.9128** y un **F1-score de 0.9477**.

Esto significa que, cuando el modelo clasificó un mensaje como spam, acertó aproximadamente en el 98.55% de los casos. Al mismo tiempo, logró detectar alrededor del 91.28% de todos los mensajes spam existentes en el conjunto de prueba.

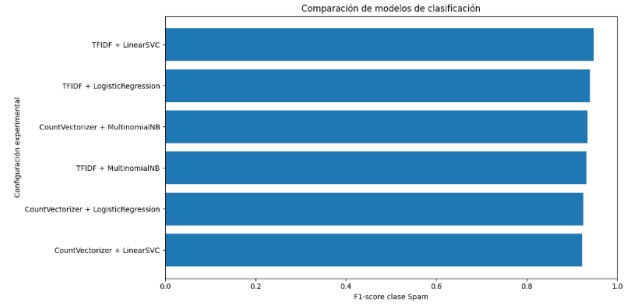


Fig. 2: Comparación del F1-score correspondiente a la clase spam para las seis configuraciones experimentales seleccionadas

La Figura 2 permite observar con mayor claridad que TF-IDF + LinearSVC presenta el valor más alto de F1-score para la clase spam, seguido por TF-IDF + Regresión Logística y CountVectorizer + Multinomial Naive Bayes.

La diferencia entre los modelos no es extremadamente grande, lo cual indica que varias configuraciones son capaces de resolver adecuadamente el problema. Sin embargo, TF-IDF + LinearSVC proporciona el mejor equilibrio entre precisión y capacidad de recuperación de mensajes spam.

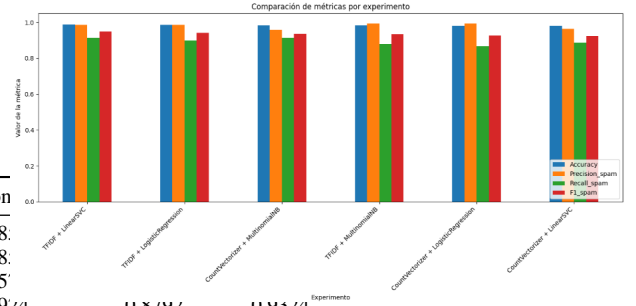


Fig. 3: Comparación de accuracy, precision, recall y F1-score para la clase spam entre las distintas configuraciones experimentales.

Esta gráfica permite complementar la Tabla 2 y visualizar que un modelo con una precisión muy elevada no necesariamente presenta el mejor recall. Por ejemplo, TF-IDF + MultinomialNB obtuvo la mayor precisión para spam, con **0.9924**, pero su recall fue menor, con **0.8792**. Esto significa que produce muy pocos falsos positivos, aunque deja sin detectar una mayor proporción de mensajes spam.

Por el contrario, TF-IDF + LinearSVC consigue mantener una precisión muy elevada de 0.9855 y, simultáneamente, incrementar el recall hasta 0.9128. Este equilibrio explica su mayor F1-score.

B. Resultados de la validación cruzada

Un aspecto importante del experimento es diferenciar entre los resultados obtenidos durante la validación cruzada y el desempeño final sobre el conjunto de prueba.

La mayor puntuación promedio de F1 macro durante la validación cruzada correspondió a **TF-IDF + Multinomial Naive**

Bayes, con un valor de **0.9699**, seguido por CountVectorizer + Multinomial Naive Bayes con **0.9690** y TF-IDF + LinearSVC con **0.9674**.

TABLE III: Mejor resultado de validación cruzada por familia experimental

Configuración	F1 macro CV	Desviación estándar
TF-IDF + MultinomialNB	0.9699	0.0053
CountVectorizer + MultinomialNB	0.9690	0.0043
TF-IDF + LinearSVC	0.9674	0.0033
CountVectorizer + LinearSVC	0.9642	0.0046
TF-IDF + Logistic Regression	0.9639	0.0040
CountVectorizer + Logistic Regression	0.9590	0.0069

Aunque TF-IDF + MultinomialNB obtuvo el mayor F1 macro promedio durante la validación cruzada, no fue el modelo con mejor F1 para spam sobre el conjunto de prueba independiente. TF-IDF + LinearSVC obtuvo un F1 spam de 0.9477 frente a 0.9324 de TF-IDF + MultinomialNB.

Este resultado no representa una contradicción. La validación cruzada se utiliza para estimar la capacidad de generalización durante el proceso de selección del modelo, mientras que el conjunto de prueba proporciona una evaluación final sobre observaciones que permanecieron completamente separadas del entrenamiento. Además, la métrica utilizada durante GridSearchCV fue F1 macro, que considera ambas clases por igual, mientras que el análisis final también examina específicamente el F1 de la clase spam.

Otro aspecto favorable para TF-IDF + LinearSVC es que presentó una desviación estándar relativamente baja durante la validación cruzada, de aproximadamente 0.0033, lo que indica un comportamiento estable entre las cinco particiones utilizadas.

VI. EFECTO DE LOS HIPERPARÁMETROS

Los resultados del GridSearch muestran que la selección de hiperparámetros produjo diferencias observables en el desempeño de los modelos.

En cinco de las seis mejores configuraciones se seleccionó una representación que incluye **unigramas y bigramas**, mediante $ngram_range = (1,2)$. Esto sugiere que incorporar secuencias de dos palabras aporta información adicional relevante para distinguir entre mensajes ham y spam.

Los bigramas permiten representar expresiones que pueden ser más discriminativas que las palabras consideradas de forma individual. En un problema de detección de spam, combinaciones frecuentes de términos promocionales, llamados a la acción o expresiones asociadas con premios y ofertas pueden proporcionar señales útiles para la clasificación.

Para el modelo seleccionado como mejor configuración, **TF-IDF + LinearSVC**, los hiperparámetros óptimos fueron:

- $C = 1$
- $max_features = None$
- $min_df = 2$
- $ngram_range = (1,2)$

El parámetro $C = 1$ proporciona un equilibrio entre la penalización de errores y la regularización del modelo. La elección

de $min_df = 2$ implica que se excluyen términos que aparecen solamente una vez en el corpus de entrenamiento, reduciendo características extremadamente raras que podrían introducir ruido.

La selección de $max_features = None$ indica que no resultó conveniente limitar artificialmente el vocabulario a las 5,000

características consideradas como alternativa experimental.

Por lo tanto, el mejor resultado se obtuvo permitiendo utilizar todas las características que cumplieran con los demás criterios de filtrado.

Finalmente, $ngram_range = (1,2)$ confirma que la combinación de unigramas y bigramas proporcionó una representación más conveniente para el clasificador.

TABLE IV: Hiperparámetros óptimos de cada modelo

Configuración	Mejores hiperparámetros
TF-IDF + LinearSVC	$C=1$, $min_df=2$, $ngram=(1,2)$, $max_features=None$
TF-IDF + Logistic Regression	$C=10$, $min_df=1$, $ngram=(1,2)$, $max_features=None$
CountVectorizer + MultinomialNB	$alpha=0.5$, $min_df=1$, $ngram=(1,2)$
TF-IDF + MultinomialNB	$alpha=0.1$, $min_df=1$, $ngram=(1,2)$
CountVectorizer + Logistic Regression	$C=10$, $min_df=2$, $ngram=(1,2)$
CountVectorizer + LinearSVC	$C=10$, $min_df=2$, $ngram=(1,2)$

Un resultado especialmente consistente es que **todas las mejores configuraciones seleccionaron unigramas y bigramas**, en lugar de trabajar exclusivamente con palabras individuales. Esto constituye evidencia experimental de que, para este conjunto de datos, la incorporación de cierto contexto local mejora o favorece el proceso de clasificación.

A. Evaluación detallada del mejor modelo

El modelo seleccionado finalmente fue **TF-IDF + LinearSVC**.

Su clasificación detallada sobre las 1,115 observaciones del conjunto de prueba fue la siguiente:

TABLE V: Reporte de clasificación del modelo TF-IDF + LinearSVC

Clase	Precision	Recall	F1-score	Observaciones
Ham	0.9867	0.9979	0.9923	966
Spam	0.9855	0.9128	0.9477	149
Accuracy	0	0	0.9865	1,115
Promedio macro	0.9861	0.9553	0.9700	1,115
Promedio ponderado	0.9865	0.9865	0.9863	1,115

El desempeño para la clase ham fue particularmente elevado, obteniendo un F1-score de **0.9923**. Para la clase spam, que representa el problema de mayor interés, se obtuvo un F1-score de **0.9477**.

El promedio macro de F1 fue de **0.9700**, lo que indica que el modelo mantiene un buen desempeño al considerar ambas categorías con igual importancia.

La diferencia entre el recall de ham, de 0.9979, y el recall de spam, de 0.9128, refleja parcialmente el desbalance original del conjunto de datos y la mayor dificultad relativa de identificar todos los casos pertenecientes a la clase minoritaria.

B. Análisis de la matriz de confusión

Para comprender con mayor detalle los errores producidos por el mejor modelo se analizó su matriz de confusión.

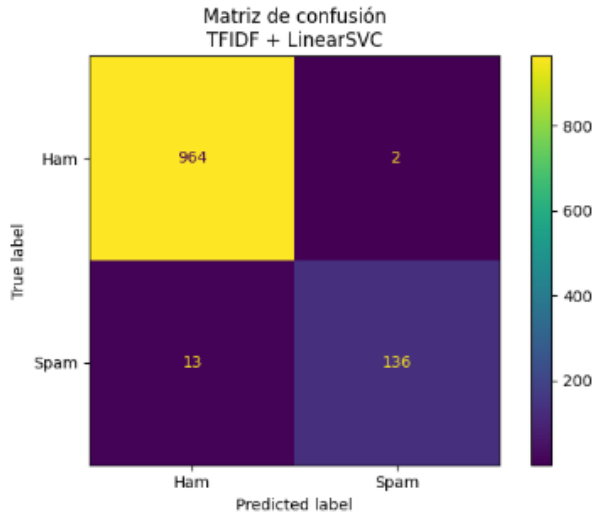


Fig. 4: Matriz de confusión correspondiente al modelo TF-IDF + LinearSVC sobre el conjunto independiente de prueba.

La matriz de confusión puede interpretarse de la siguiente manera:

Clase real	Predicción Ham	Predicción Spam
Ham	964	2
Spam	13	136

De los **966 mensajes ham** presentes en el conjunto de prueba, el modelo clasificó correctamente **964**, mientras que solamente **2 mensajes legítimos fueron incorrectamente identificados como spam**.

Por otra parte, de los **149 mensajes spam**, el modelo identificó correctamente **136**, mientras que **13 mensajes spam fueron clasificados erróneamente como ham**.

En total se produjeron solamente **15 errores de clasificación entre 1,115 observaciones**, lo que concuerda con el accuracy de aproximadamente 98.65%.

Desde una perspectiva práctica, los dos tipos de errores tienen implicaciones diferentes.

Los **falsos positivos**, correspondientes a mensajes ham clasificados como spam, son especialmente sensibles porque podrían provocar que un mensaje legítimo sea bloqueado o enviado a una carpeta de correo no deseado. El modelo produjo únicamente dos casos de este tipo, lo que explica su alta precisión para la clase spam.

Por otro lado, los **falsos negativos**, correspondientes a mensajes spam clasificados como ham, representan mensajes no deseados que el sistema no consiguió detectar. Se observaron 13 casos de este tipo. Por esta razón, aunque el desempeño general del modelo es elevado, existe todavía margen para mejorar el recall de la clase spam.

El comportamiento observado refleja un modelo relativamente conservador al etiquetar mensajes como spam: cuando asigna dicha categoría suele hacerlo correctamente, pero una pequeña proporción de mensajes spam permanece sin detectar.

VII. DISCUSIÓN

Los resultados obtenidos muestran que el desempeño en clasificación de textos depende de la interacción entre el método de representación, el algoritmo de clasificación y sus hiperparámetros. Las diferencias encontradas entre las seis configuraciones demuestran que no basta con seleccionar un algoritmo de aprendizaje de forma aislada; la representación del texto tiene también un papel fundamental.

En términos del conjunto independiente de prueba, **TF-IDF combinado con LinearSVC fue la configuración con mejor desempeño global**, alcanzando un accuracy de 0.9865 y un F1-score de 0.9477 para la clase spam.

El buen comportamiento de TF-IDF puede relacionarse con su capacidad para ponderar la relevancia de los términos. A diferencia de CountVectorizer, que utiliza directamente las frecuencias de aparición, TF-IDF reduce la influencia de palabras presentes en una gran cantidad de documentos y otorga mayor importancia relativa a términos que pueden aportar información discriminativa.

Este comportamiento resulta particularmente útil en mensajes de texto, donde determinadas palabras o expresiones asociadas con promociones, premios, solicitudes urgentes o llamados a la acción pueden contribuir significativamente a diferenciar mensajes spam de comunicaciones legítimas.

También se observó que todas las configuraciones óptimas utilizaron *ngram_range=(1,2)*. Este resultado indica que la incorporación de **bigramas**, además de unigramas, fue sistemáticamente seleccionada durante la optimización. Por lo tanto, el contexto formado por combinaciones de dos palabras proporciona información útil que no necesariamente se obtiene cuando cada término se analiza únicamente de forma independiente.

Respecto a los algoritmos, LinearSVC mostró un comportamiento especialmente competitivo. Los modelos SVM lineales suelen adaptarse adecuadamente a representaciones textuales debido a que estas generan espacios de características de alta dimensionalidad y matrices dispersas. En este experimento, dicha combinación permitió lograr un equilibrio favorable entre precisión y recall.

Sin embargo, también es relevante analizar el desempeño de **Multinomial Naive Bayes**. La configuración TF-IDF + MultinomialNB obtuvo el mayor F1 macro durante la validación cruzada, con 0.9699, y la mayor precisión para spam sobre el conjunto de prueba, con 0.9924. Esto indica que fue extremadamente preciso cuando decidió asignar la categoría spam.

No obstante, su recall fue de 0.8792, inferior al 0.9128 alcanzado por LinearSVC. En otras palabras, MultinomialNB fue más conservador al identificar spam y produjo menos falsos positivos, pero dejó sin detectar una mayor cantidad de mensajes pertenecientes a dicha clase.

Esta diferencia demuestra la importancia de no seleccionar un modelo utilizando una sola métrica. En un sistema real, la elección del modelo podría depender del costo asociado a cada tipo de error. Si el principal objetivo fuera evitar que mensajes legítimos sean clasificados como spam, un modelo con una precisión extremadamente alta podría resultar atractivo. Por el contrario, si fuera prioritario detectar la mayor cantidad posible de spam, sería necesario favorecer un mayor recall.

En este trabajo se consideró el **F1-score para spam** como una medida especialmente relevante porque resume el equilibrio entre precisión y recall. Bajo este criterio, TF-IDF + LinearSVC presentó el mejor resultado.

La Regresión Logística también mostró un desempeño competitivo, principalmente al utilizar TF-IDF. La configuración TF-IDF + Logistic Regression alcanzó un F1 de 0.9404 para spam, constituyendo el segundo mejor resultado en esta métrica. Esto demuestra que modelos lineales relativamente simples pueden conseguir resultados muy elevados cuando se combinan con una representación textual adecuada y una optimización sistemática de hiperparámetros.

Por otro lado, las configuraciones con CountVectorizer presentaron generalmente un desempeño ligeramente inferior a sus alternativas basadas en TF-IDF, aunque la diferencia no fue uniforme. Un caso destacable fue CountVectorizer + MultinomialNB, que alcanzó un recall de 0.9128, igual al obtenido por el modelo ganador, aunque con una precisión inferior de 0.9577 y, consecuentemente, un F1 menor de 0.9347.

Este resultado confirma que no existe una única técnica universalmente superior para todos los criterios. La selección final depende de la interacción entre representación, modelo, hiperparámetros y métrica considerada.

Otro hallazgo relevante corresponde a la diferencia entre la selección mediante validación cruzada y el resultado sobre el conjunto de prueba. TF-IDF + MultinomialNB obtuvo el mayor F1 macro promedio durante validación cruzada, mientras que TF-IDF + LinearSVC presentó el mayor F1 específico para spam sobre los datos de prueba.

Esta diferencia es esperable dentro de un proceso experimental. La validación cruzada proporciona una estimación del desempeño promedio utilizando múltiples particiones de entrenamiento, mientras que el conjunto de prueba representa una muestra independiente empleada únicamente en la evaluación final. Además, las métricas comparadas no son exactamente equivalentes: el GridSearch fue optimizado mediante F1 macro, mientras que para la selección final se prestó especial atención al F1 correspondiente específicamente a la clase spam.

La utilización de GridSearchCV permitió evaluar **96 configuraciones de hiperparámetros y 480 ajustes mediante validación cruzada**, aportando mayor rigor al proceso de selección que el uso de una única configuración arbitraria. Este enfoque permite sustentar la elección final mediante evidencia experimental y no únicamente por decisiones subjetivas sobre los parámetros.

En términos generales, los resultados indican que los métodos clásicos de aprendizaje automático continúan siendo altamente efectivos para problemas de clasificación textual cuando se combinan con representaciones apropiadas como TF-IDF y una selección sistemática de hiperparámetros.

VIII. CONCLUSIONES

En el presente trabajo se desarrolló un diseño experimental para comparar diferentes combinaciones de métodos de vectorización, algoritmos de clasificación e hiperparámetros aplicados a la detección automática de mensajes spam.

El diseño incluyó seis familias experimentales construidas mediante la combinación de CountVectorizer y TF-IDF con Regresión Logística, Multinomial Naive Bayes y LinearSVC. Mediante GridSearchCV se evaluaron un total de 96 configuraciones distintas y 480 ajustes derivados de la validación cruzada estratificada de cinco particiones.

Los resultados demuestran que la selección del método de representación y de los hiperparámetros influye directamente sobre la capacidad predictiva de los modelos. En particular, todas las configuraciones óptimas seleccionaron representaciones que incorporaban unigramas y bigramas, lo que sugiere que la información contextual formada por secuencias de dos palabras aporta valor para la identificación de mensajes spam.

La configuración que presentó el mejor desempeño sobre el conjunto independiente de prueba fue **TF-IDF + LinearSVC**, utilizando $C=1$, $min_df=2$, $ngram_range=(1,2)$ y sin una restricción específica en el número máximo de características.

Este modelo alcanzó un **accuracy de 98.65%**, una **precisión de 98.55%**, un **recall de 91.28%** y un **F1-score de 94.77% para la clase spam**.

El análisis de la matriz de confusión mostró que, de 1,115 observaciones del conjunto de prueba, solamente se produjeron 15 clasificaciones incorrectas. De los 966 mensajes ham, 964 fueron correctamente identificados y únicamente 2 fueron clasificados erróneamente como spam. En el caso de los 149 mensajes spam, 136 fueron correctamente detectados y 13 fueron clasificados como mensajes legítimos.

Estos resultados muestran que el modelo presenta una tasa muy reducida de falsos positivos y una alta capacidad para distinguir mensajes legítimos de spam. Sin embargo, los falsos negativos observados indican que existe todavía margen para incrementar la sensibilidad hacia la clase spam.

Otro hallazgo importante fue que el modelo con mayor puntuación promedio durante la validación cruzada no coincidió exactamente con el modelo que obtuvo el mayor F1 para spam sobre el conjunto de prueba. TF-IDF + Multinomial Naive Bayes alcanzó el mayor F1 macro durante validación cruzada, mientras que TF-IDF + LinearSVC obtuvo el mejor equilibrio entre precisión y recall para spam en la evaluación final. Este resultado destaca la importancia de utilizar un conjunto de prueba independiente y analizar múltiples métricas antes de seleccionar definitivamente un modelo.

En conclusión, el diseño experimental permitió demostrar que la combinación de **TF-IDF, unigramas y bigramas, y un**

clasificador SVM lineal adecuadamente regularizado constituye la alternativa más favorable entre las configuraciones analizadas para este problema de clasificación textual.

Como trabajo futuro, el experimento podría ampliarse mediante técnicas más avanzadas de representación semántica, tales como Word2Vec, FastText, Sentence Transformers o modelos preentrenados basados en arquitecturas Transformer como BERT. Asimismo, sería posible incorporar técnicas específicas para el tratamiento del desbalance de clases, ampliar el espacio de búsqueda de hiperparámetros y evaluar el modelo sobre conjuntos de datos externos para analizar su capacidad de generalización en contextos diferentes.

REFERENCES

[learn Developers, 2026] learn Developers, S. (2026). Scikit-learn: Machine learning in python.